



CUBELEARN · IBAW LUZERN

MongoDB Aufgaben

Die fünf Übungen aus Kapitel 6 von „Pro MERN Stack 2“,
gelöst und erklärt — verknüpft mit einem eigenen Issue-
Tracker-Backend auf MongoDB Atlas.

Worum es hier geht

Dieses Dokument beantwortet die fünf Übungsfragen aus Kapitel 6 (MongoDB) von „Pro MERN Stack 2“. Das Buchprojekt ist ein **Issue Tracker**; das `employees`-Beispiel mit John und Jane Doe ist nur der Spielplatz in der Mongo-Shell, an dem die CRUD-Konzepte gezeigt werden. Die Übungen selbst sind **themenneutral** — sie fragen nach Mechanik (Cursor, Filter, Indizes, Verbindung, Write-Pattern), nicht nach einem bestimmten Datenmodell.

Ich beantworte jede Frage zweifach: einmal die Buch-Antwort (was die Mongo-Shell bzw. der native Driver tun), und einmal in Bezug auf mein eigenes Issue-Tracker-Backend, das ich für den Nachbereitungsauftrag 4.1 gebaut habe — Express + Mongoose gegen eine MongoDB-Atlas-Datenbank. So sieht man die Theorie und den laufenden Code nebeneinander.

HINWEIS ZUR UMGEBUNG

Das Buch nutzt oft den **nativen MongoDB-Driver** (`db.collection('x').find()`). Mein Backend nutzt **Mongoose**, eine Schicht darüber, die Schema-Validierung, Default-Werte und bequeme Methoden mitbringt. Wo Mongoose etwas anders macht als das Buch, steht es dabei.

1 · Cursor

Was ist ein Cursor und welche Methoden hat er?

2 · CRUD

`$exists`, ist der Filter JSON, `$unset`, die `1` beim Index.

3 · Schema-Init

Mongo-Shell vs. Node.js, welche Indizes fehlen?

4 · Connection

Verbindungsverlust und eine Million Dokumente.

5 · Write-Pattern

Warum nach dem Schreiben nochmal lesen?

Der Cursor und seine Methoden

Frage aus dem Buch: Lass dir über die Shell eine Liste der Methoden anzeigen, die auf dem Cursor-Objekt verfügbar sind.

Was ein Cursor überhaupt ist

Das Wort „Cursor“ ist irreführend — es hat nichts mit der blinkenden Textmarke zu tun. Ich denke an eine Bibliothekarin: Ich sage „alle Bücher von Murakami“, und sie hat zwei Möglichkeiten. Entweder schleppt sie alle 30 Bücher in einer Kiste an meinen Tisch (bei 30.000 Büchern bricht der Tisch zusammen), oder sie sagt „ich hab sie gefunden, hier ist mein Lesezeichen, sag mir wann ich dir das nächste bringe“. Dieses Lesezeichen **ist** der Cursor.

Der Cursor ist also nicht das Ergebnis, sondern der Zeiger darauf. Der MongoDB-Server hält die Position; ich kann sortieren, weiterspulen, begrenzen — alles, ohne dass die Daten schon zu mir geschickt wurden.

`find()` gibt einen Cursor zurück, kein Array.

Die Antwort: Methoden auflisten

In der Mongo-Shell gibt es zwei Wege, an die Liste der Cursor-Methoden zu kommen:

```
// Weg 1: Hilfe direkt auf dem Cursor aufrufen
db.employees.find().help()

// Weg 2: Cursor in eine Variable, dann TAB-TAB drücken
var c = db.employees.find()
c.    // (Tab zweimal) -> listet alle verfügbaren Methoden
```

Beide stützen sich darauf, dass die Shell eine JavaScript-Umgebung ist: doppeltes Tab nach dem Punkt triggert die Auto-Vervollständigung und zeigt alle Methoden.

Die wichtigsten Cursor-Methoden

- `.sort({ feld: 1 })` — Reihenfolge festlegen (1 = aufsteigend, -1 = absteigend).
- `.limit(n)` — höchstens n Dokumente.
- `.skip(n)` — die ersten n überspringen (zusammen mit limit = Pagination).
- `.toArray()` — alle Treffer in ein Array laden.
- `.count()` / `.countDocuments()` — wie viele Treffer.
- `.hasNext()` / `.next()` — Stück für Stück durchgehen, statt alles auf einmal.
- `.forEach(fn)` — über alle Treffer iterieren.
- `.pretty()` — die Ausgabe lesbar formatieren.

Wo das in meinem Backend vorkommt

In meinem Issue-Tracker sehe ich den Cursor nie direkt — Mongoose schluckt ihn weg und gibt mir gleich das Array. Trotzdem nutze ich Cursor-Methoden:

```
// controllers/issueController.js – listIssues()
const issues = await Issue.find(filter).sort({ id: 1 });
```

IM BUCH SIEHT'S SO AUS

Der Autor arbeitet mit dem nativen Driver und bekommt den Cursor explizit:

`db.collection('users').find({}).skip(10).limit(20)` — und muss selbst `.toArray()` aufrufen. Mongoose macht dieses `.toArray()` automatisch im Hintergrund.

WAS ICH MIR GEMERKT HABE

Cursor ist das Lesezeichen, nicht das Ergebnis. Mongoose versteckt ihn meistens und gibt mir gleich das Array. Den expliziten Cursor (mit `.cursor()` und `for await`) brauche ich erst, wenn ich sehr viele Dokumente streamen will, ohne dass der RAM explodiert.

\$exists, JSON, \$unset und die 1

Fragen aus dem Buch: Wie hole ich alle Dokumente mit einem bestimmten Feld? Ist die Filter-Spezifikation ein JSON? Wie lösche ich ein einzelnes Feld aus einem Dokument? Was bedeutet die `1` bei `createIndex({ field: 1 })` ?

\$exists — alle Dokumente, die ein Feld haben

Die Frage „alle Mitarbeiter mit Middle Name“ geht nicht über „ist gleich“, sondern über „ist diese Schublade überhaupt befüllt?“ — dafür gibt es `$exists` :

```
// Alle Employees, bei denen ein middle name gesetzt ist
db.employees.find({ "name.middle": { $exists: true } })
```

Wichtig: `$exists: true` heißt „das Feld kommt im Dokument vor“ — nicht „der Wert ist nicht null“. Das ist ein feiner, aber wichtiger Unterschied.

Ist die Filter-Spezifikation ein JSON?

Kurze Antwort: **nein**. Es sieht aus wie JSON, ist aber ein JavaScript-Objekt. Drei Indizien:

- Die Keys sind **unquotiert** (`status` , nicht `"status"`) — JSON verlangt Anführungszeichen.
- Ich kann **Variablen und Konstanten** als Werte einsetzen — JSON kennt keine Variablen.
- Ich kann **echte Objekte** wie `Date` oder `ObjectId` reinpacken — JSON kann nur Strings, Zahlen, Booleans.

Bevor der Filter an die DB geht, wird er vom Driver in **BSON** (Binary JSON) serialisiert. BSON kann `ObjectId` , `Date` und mehr — JSON nicht.

```
// Aus meinem Backend – der Filter wird Stück für Stück gebaut:
const filter = {};
if (req.query.status) filter.status = req.query.status;
if (req.query.owner) filter.owner = req.query.owner;
await Issue.find(filter);
```

Genau das wäre mit echtem JSON nicht möglich — ich nutze hier Variablen und baue das Objekt zur Laufzeit zusammen.

\$unset — ein Feld komplett entfernen

Ein versehentlich gesetztes Feld wieder loswerden:

```
db.employees.updateOne(
  { _id: someId },
  { $unset: { "name.middle": "" } } // Wert egal, Feld wird gelöscht
)
```

Die Pointe: `$unset` ist **nicht** dasselbe wie auf `null` setzen. `$unset` reißt das Etikett ab; auf `null` setzen schreibt „leer“ aufs Etikett — das Feld bleibt da.

Was bedeutet die 1 bei createIndex?

Telefonbuch-Analogie: ein Telefonbuch ist nach Nachname sortiert. Ich kann vorwärts blättern (A → Z) oder rückwärts (Z → A). `1` heißt **aufsteigend**, `-1` heißt **absteigend**. Andere gültige Werte sind spezielle Typen wie `'text'` (Volltext-Index) oder `'2dsphere'` (Geo-Index).

```
// Aus meinem Backend – models/Issue.js
issueSchema.index({ status: 1 }); // aufsteigend
issueSchema.index({ created: 1 });
```

Bei einem Single-Field-Index ist die Richtung egal (MongoDB kann den Index in beide Richtungen lesen). Wichtig wird sie bei Compound-Indizes: dort muss die Richtung zur `.sort()`-Richtung der Query passen, sonst kann der Index für die Sortierung nicht genutzt werden.

IM BUCH SIEHT'S SO AUS

Identische Syntax — `$exists`, `$unset` und die Index-Richtung sind **Datenbank-Operatoren**, kein Mongoose-Spezifikum. Mongoose reicht sie 1:1 an MongoDB durch.

WAS ICH MIR GEMERKT HABE

`$exists` = „Feld kommt vor“, nicht „Wert ist nicht null“. Der Filter ist ein JS-Objekt (kein JSON), deshalb darf ich Variablen, Dates und ObjectIds reinpacken. `$unset` löscht ein Feld ganz, `$set` setzt es. `1` / `-1` sind die Sortierrichtung des Index.

Mongo-Shell vs. Node.js, und welche Indizes fehlen

Fragen aus dem Buch: Vor- und Nachteile von Mongo-Shell-Skript gegenüber Node.js-Skript für die Initialisierung? Welche weiteren Indizes wären sinnvoll?

Init-Skript: Mongo-Shell vs. Node.js

Beide Wege legen Testdaten und Indizes an — die Frage ist „welches Werkzeug für welchen Kontext“.

Mongo-Shell-Skript	Node.js + Mongoose
Läuft überall, wo die Shell installiert ist. Kein Setup, kein npm.	Eine konsistente Welt: gleiche Schemas, gleiche Connection-Logik wie die App.
Gut für DB-Admins und kleine One-Shot-Operationen.	Gut für komplexe Apps mit vielen Models — keine doppelte Pflege.
Keine npm-Pakete, keine ENV-Variablen, keine Mongoose-Schemas.	Braucht eine Node-Umgebung mit installierten Modulen.

Für mein Backend nutze ich **Node.js + Mongoose** (das Seed-Skript `scripts/seed.js`), weil Schemas und Connection bereits da sind. Der Trade-off — ich brauche Node im Setup — ist kein Problem, weil das Backend sowieso Node-basiert ist.

```
// scripts/seed.js – das Node.js-Pendant zu init.mongo.js aus dem Buch
await Issue.deleteMany({});
await Issue.insertMany(sampleIssues);
await Counter.findOneAndUpdate(
  { _id: 'issues' },
  { $set: { current: sampleIssues.length } },
  { upsert: true }
);
```

Welche zusätzlichen Indizes wären sinnvoll?

Der Buch-Hinweis zielt auf eine **Suchleiste**. Sobald ich Issues per Freitext durchsuchen will (z. B. im Titel), ist ein **Text-Index** der richtige Typ — er erlaubt Volltext-Suche mit Wort-Stemming und Relevanz-Score, statt linear mit Regex durch die Collection zu laufen.

```
// Wäre sinnvoll, sobald eine Suchleiste dazukommt:
issueSchema.index({ title: 'text' });

// Dann ließe sich so suchen:
await Issue.find({ $text: { $search: 'login button' } });
```

Daneben sind die Filter-Felder gute Index-Kandidaten — in meinem Backend habe ich deshalb `status`, `owner` und `created` bereits indiziert, weil genau danach gefiltert und sortiert wird.

IM BUCH SIEHT'S SO AUS

Der Autor nutzt für sein kleines Demo das Shell-Skript, gibt aber zu, dass für ein produktives Projekt Node.js besser wäre. Text-Indizes erwähnt er nur kurz als weiterführendes Thema — genau dort, wo eine echte Suchleiste anfängt.

WAS ICH MIR GEMERKT HABE

Shell-Skript = portabel, kein Setup, gut für Admins. Node.js = konsistent mit der App, gut für komplexe Projekte. Ein Index ist eine Design-Entscheidung, kein späteres Performance-Tuning — und für Suchleisten gibt es den eigenen Typ Text-Index.

Verbindungsverlust und eine Million Dokumente

Fragen aus dem Buch: Was passiert, wenn die DB-Verbindung verloren geht (kurz und lang)? Wie würde ich mit einer Million Dokumente umgehen, statt `toArray()` zu nutzen?

Was bei Verbindungsverlust passiert

Die Connection ist in Wahrheit ein **Pool** — ein Team von offenen Leitungen zur DB, das sich selbst verwaltet. Daraus folgt:

- **Kurzer Ausfall (< ~30 Sek):** Der Driver merkt den Verlust intern, versucht automatisch zu reconnecten und führt wartende Queries danach aus. Der Server überlebt das ohne Neustart.
- **Langer Ausfall:** Wartende Queries laufen in einen Timeout und scheitern mit einem Fehler. Beim nativen Driver muss man danach unter Umständen den Server neu starten; die Stellschrauben heißen `reconnectTries` und `reconnectInterval` bzw. bei neueren Treibern die Timeout-Optionen.

Mein Connection-Setup

```
// config/db.js
async function connectDB() {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI);
    console.log(`MongoDB verbunden: ${conn.connection.host}`);
  } catch (err) {
    console.error(`MongoDB-Verbindungsfehler: ${err.message}`);
    process.exit(1); // bewusst hart beim initialen Connect
  }
}
```

Meine einzige Verantwortung ist der **initiale** Connect mit Hard-Exit. Den laufenden Reconnect bei kurzen Aussetzern übernimmt Mongoose mit sinnvollen Defaults selbst — dafür schreibe ich keine eigene Logik.

Eine Million Dokumente: was passiert mit `toArray()`?

`.find().toArray()` will die **ganze** Bibliothek auf einmal nach Hause tragen — bei einer Million Dokumenten sprengt das den RAM (Out-of-Memory). Die robuste Lösung ist Streaming mit einem Cursor: dann lebt nur **ein** Dokument gleichzeitig im Speicher.

```
// Statt alles auf einmal – Stück für Stück streamen:
const cursor = Issue.find({}).cursor();
for await (const issue of cursor) {
  // genau ein Dokument im RAM, egal wie viele es insgesamt sind
}
```

Alternativen sind `.limit(n)` (nur die ersten n) und `.skip()` für Pagination. Wichtig: `.skip()` wird langsamer, je tiefer man geht (die DB muss die übersprungenen Dokumente trotzdem lesen). Bei sehr großen Datenmengen ist Cursor-basierte Pagination mit `{ _id: { $gt: lastSeenId } }` effizienter.

IM BUCH SIEHT'S SO AUS

Der Autor empfiehlt für große Datasets ausdrücklich Cursor-basierte Iteration mit `hasNext()` / `next()`. Die `for await`-Variante ist dieselbe Idee in moderner Async-Iterator-Syntax.

WAS ICH MIR GEMERKT HABE

Der Connection-Pool und der Auto-Reconnect kommen gratis — meine Aufgabe ist nur der harte initiale Connect. `toArray()` auf riesige Collections ist ein latenter Bug; die Lösung heißt `.cursor()` mit `for await`.

Warum nach dem Schreiben nochmal lesen?

Frage aus dem Buch: Hätte man die `_id` einfach zum übergebenen Objekt hinzufügen und das zurückgeben können, statt nochmal `findOne()` zu machen?

Trust but verify

Auf den ersten Blick wirkt das nochmalige Lesen wie unnötiger Overhead — die DB hat doch gerade bestätigt, dass der Write erfolgreich war. Der blinde Fleck: Beim Insert kann die DB **Default-Werte** gesetzt haben (`created` , `status: 'New'`), die im Original-Objekt nicht standen. Gebe ich nur das Original zurück, verschicke ich ein unvollständiges Objekt an den Client.

Deshalb: einen Round-Trip extra investieren und das lesen, **was wirklich in der DB steht** — die Datenbank ist die Source of Truth. Das ist keine Paranoia, sondern Konsequenz.

Wie Mongoose das löst

Mongoose hat hier Glück: `create()` gibt mir das fertige Dokument inklusive aller Defaults direkt zurück — ich muss **nicht** nochmal lesen.

```
// controllers/issueController.js - createIssue()
issue.id = await getNextSequence('issues');
const result = await Issue.create(issue); // fertiges Doc inkl. Defaults
res.status(201).json(result);
```

Beim Update gilt dasselbe über die Option `{ new: true }` — sie gibt mir das Dokument **nach** der Änderung zurück, atomar in einem Round-Trip.

Der Sequence-Counter aus dem Buch

Das verwandte Problem: zwei gleichzeitige Inserts könnten beide dieselbe id vergeben (Race Condition). Die Buch-Lösung ist ein eigener Counter mit `$inc` — wie eine Nummernziehmaschine an der Wursttheke: atomar, zwei gleichzeitige Kunden bekommen garantiert nie dieselbe Nummer.

```
// controllers/issueController.js - getNextSequence()
async function getNextSequence(name) {
  const result = await Counter.findOneAndUpdate(
    { _id: name },
    { $inc: { current: 1 } }, // atomares Hochzählen
    { new: true, upsert: true }
  );
  return result.current;
}
```

Würde ich die nächste id über „lesen, +1, schreiben“ bilden, hätte ich genau die Race Condition, die `$inc` verhindert. Weil `findOneAndUpdate` mit `$inc` eine einzige atomare Operation ist, bekommen parallele Aufrufe garantiert eindeutige, fortlaufende Zahlen.

IM BUCH SIEHT'S SO AUS

Beim nativen Driver gibt `insertOne` nur die Insert-Statistik zurück, nicht das Dokument — der Autor braucht den expliziten zweiten `findOne`. In meinem Mongoose-Setup ist dieses Problem schon weggekapselt.

WAS ICH MIR GEMERKT HABE

„Trust but verify“: beim nativen Driver lese ich nach dem Schreiben nochmal, weil die DB die Source of Truth ist. Mongoose erspart mir das, weil `create()` und `findOneAndUpdate({ new: true })` das fertige Dokument zurückgeben. `$inc` löst Race Conditions gratis, weil es atomar ist.

Aha-Momente auf einer Seite

Cursor — ist nicht das Ergebnis, sondern das Lesezeichen. Mongoose versteckt ihn meistens und gibt gleich das Array zurück.

Filter — ist kein JSON, sondern ein JS-Objekt. Deshalb sind Variablen, ObjectIds, Dates und unquoted Keys erlaubt.

\$exists — heißt „Feld kommt im Dokument vor“, nicht „Wert ist nicht null“.

\$set / \$unset — `$set` setzt ein Feld, `$unset` entfernt es komplett (anders als auf null setzen).

\$inc — zählt einen Counter atomar hoch und löst damit Race Conditions gratis.

Index-Richtung — `1` aufsteigend, `-1` absteigend. Bei Compound-Indizes muss sie zur Sort-Richtung passen.

Text-Index — `{ field: 'text' }` für echte Suchleisten statt langsamer Regex-Suche.

Connection — der Pool und der Auto-Reconnect kommen gratis. Meine Aufgabe ist nur der harte initiale Connect.

Million Dokumente — `toArray()` sprengt den RAM. `.cursor()` mit `for await` hält nur ein Dokument gleichzeitig im Speicher.

Write-Pattern — beim nativen Driver „trust but verify“ (nochmal lesen); Mongoose gibt das fertige Dokument direkt zurück.